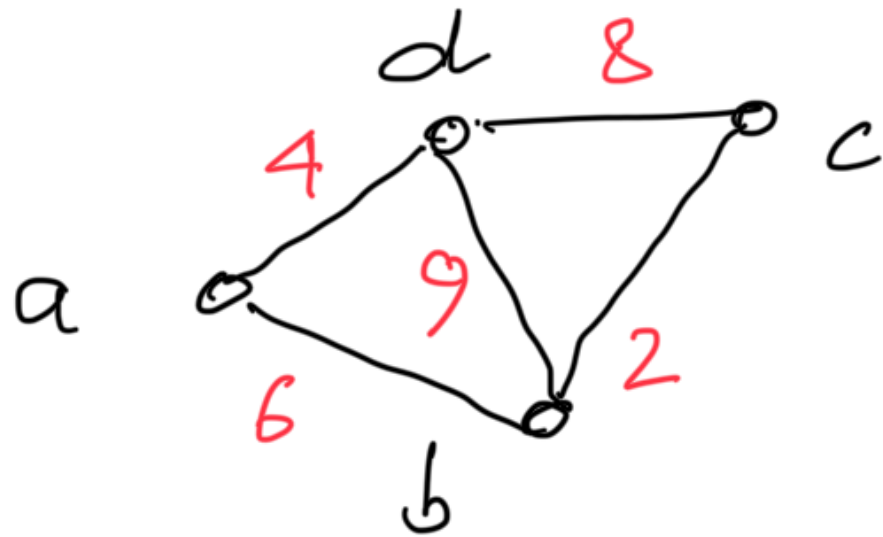


Finding a minimum spanning Tree
of a given undirected graph $G = (V, E, W)$



How many spanning
trees are there for
a given graph?

Refer to Cayley's theorem
for a complete graph

The naive approach would consider all
possible spanning trees and output one

least wt spanning tree as MST

A DFS outputs a spanning tree that is the set of edges whereby we visited a new / revisited vertex

$\Rightarrow$ for any vertex $V - \{\text{starting vertex}\}$ there is unique edge that is used to visit it for the first time (and also generates a DFS numbering).

Can we generate the MST using a greedy approach to DFS (everytime we choose a new neighbor, we pick the least weighted neighbor)

Or find a counterexample

Here is another possibility

$T \leftarrow \emptyset$ { a subset of edges }

For $i = 1$ to $n-1$ do

Greedy choice algorithm { let e_i^m be the minimum wt edge *
not considered till now

If $T \cup \{e_i^m\}$ doesn't create a
cycle with T

then $T \leftarrow T \cup \{e_i^m\}$

Output T

* We can use a heap or we can

do an initial sorting of the edge wts
in increasing order

- Clearly T is cycle free ~

- But is T the minimum ST?

Concerns: We are not discarding any
edge added to T

Why will our local greedy choices
lead to an overall global min.?

The general optimization framework

We have a set S of elements $\{e_1, e_2, \dots, e_m\}$
which have weights $w_1, w_2, \dots, w_m$

We also have a constraint on the feasible / independent subset of edges from 2^S , say $F \subseteq 2^S$

Goal : To choose the min wt subset among F

Change to Maximum wt

sum of wts of the elements in the subset

In the case of MST
 F is the family of subsets of all forests
(so cycles are allowed)

Since wts > 0 , any Maximum Spanning Forest will be (connected) spanning Tree

Consider - the version of the previous algorithm for the maximization by picking the maximum wt in every iteration or presorting in decreasing order of wts.

The algorithm for MST

- Sort m edges in decreasing order
 $O(m \log m)$

- What is the time to add an edge



check if it
forms a cycle



Does the endpoints
(u, w) belong to
same tree or not

- Same tree $\Rightarrow$ cycle, can't add
- different tree : can add

Find operations
of u, w

Implementation
structure

using union Find-Set
at most m finds
 $m-1$ unions

if not, can
add

Add by joining
the two trees

Union of
the two
subsets

+ Time to sort : $O(m \log n)$

This algorithm is called Kruskal's algorithm

The other well known algorithms

- Prim's Algorithm

- Borůvka "

Prim's algorithm grows exactly one Tree and adds an edge (u, w) of minimum weight if it doesn't induce a cycle

So implementation is easier since there is only one tree - Find is much easier as we will only keep an array

- of vertices that maintains a flag of in-tree/not-in-tree and exactly one vertex gets added - union is $O(1)$
- Find also $O(1)$

So it is dominated by initial sorting

$O(m \log m + m + n)$

sorting $\nearrow$ m $\nearrow$ n
Finds Union